

Not All Models Suit Expert Offloading: On Local Routing Consistency of Mixture-of-Expert Models

Jingcong Liang
Fudan University
jcliang22@m.fudan.edu.cn

Siyuan Wang
University of Southern California
sw_641@usc.edu

Miren Tian & Yitong Li & Duyu Tang
Huawei Technologies Ltd.
tianmiren1,liyitong3,tangduyu@huawei.com

Zhongyu Wei
Fudan University
zywei@fudan.edu.cn

Abstract

Mixture-of-Experts (MoE) enables efficient scaling of large language models (LLMs) with sparsely activated experts during inference. To effectively deploy large MoE models on memory-constrained devices, many systems introduce *expert offloading* that caches a subset of experts in fast memory, leaving others on slow memory to run on CPU or load on demand. While some research has exploited the locality of expert activations, where consecutive tokens activate similar experts, the degree of this **local routing consistency** varies across models and remains understudied. In this paper, we propose two metrics to measure local routing consistency of MoE models: (1) **Segment Routing Best Performance (SRP)**, which evaluates how well a fixed group of experts can cover the needs of a segment of tokens, and (2) **Segment Cache Best Hit Rate (SCH)**, which measures the optimal segment-level cache hit rate under a given cache size limit. We analyzed 20 MoE LLMs with diverse sizes and architectures and found that models that apply MoE on every layer and do not use shared experts exhibit the highest local routing consistency. We further showed that domain-specialized experts contribute more to routing consistency than vocabulary-specialized ones, and that most models can balance between cache effectiveness and efficiency with cache sizes approximately 2x the active experts. These findings pave the way for memory-efficient MoE design and deployment without compromising inference speed. We publish the code for replicating experiments at <https://github.com/ljcleo/moe-lrc>.

1 Introduction

Mixture-of-Experts (MoE) is a widely adopted model architecture in many large language models (LLMs) that enables efficient model size scaling through sparse activation (Fedus et al., 2022; Jiang et al., 2024; Dai et al., 2024; Abdin et al., 2024). MoE models replace dense feed-forward networks (FFNs) with multiple expert modules, with only a subset activated during inference. However, the vanilla implementation requires all experts to be loaded into memory, restricting its application on memory-constrained devices such as mobile phones. To address this limitation, the *expert offloading* technique has been proposed to allow partial loading of expert modules during inference (Eliseev & Mazur, 2023; Hwang et al., 2024; Yi et al., 2025). Specifically, expert offloading caches a subset of experts in fast memory (e.g., GPU memory) based on predefined heuristics, while storing remaining experts in slower but larger-capacity storage (e.g., CPU memory or disk). During inference, especially in the decoding stage, if a token activates an expert not cached in fast memory, the system either computes the expert forward results with CPU and slow memory (CPU offload) (Kamahori et al.,

2024; Tang et al., 2024), or unloads a cached expert according to specific rules such as Least Recently Used (LRU), and replaces it with the demanded expert (on-demand loading) (Kong et al., 2024; Zhong et al., 2025).

However, frequent CPU offloads or on-demand loading within a short period can significantly degrade the efficiency of the expert offloading system and slow down inference, particularly when processing lengthy contexts with inevitable topic shifts. Prior research has focused on optimizing the design of the expert offloading system, aiming to strategically select which experts to cache in a given context to maximize cache hit rates. Among them, some observed and exploited the locality of expert activations, where similar routing choices appear within a consecutive segment of tokens, thereby minimizing the need for CPU offloads and on-demand loading (Eliseev & Mazur, 2023; Xue et al., 2024b; Zhang et al., 2025). This is especially beneficial during the decoding phase, where tokens are generated one after another.

Nevertheless, not all MoE models exhibit such continuous routing patterns uniformly, and the degree or frequency of this phenomenon varies across models. Understanding this variance may help design MoE architectures that are friendly to expert offloading systems and vice versa. In this work, we investigate the degree of this inherent consecutive routing property, which we term **local routing consistency**, of different MoE-based LLMs to explore their potential effectiveness in segment-based expert routing or caching. Figure 1 illustrates how different levels of local routing consistency reflect different routing patterns. Specifically, we propose two metrics that quantitatively reflect the local routing consistency of a specific model. (1) **Segment Routing Best Performance (SRP)** measures how effectively a segment router that selects a fixed group of experts for all tokens in a segment can approximate the original router’s decisions. SRP not only reflects local routing consistency *without parameters other than segment length*, but also enables analyzing activation patterns of *individual experts*. (2) **Segment Cache Best Hit Rate (SCH)** represents the highest cache hit rate by any expert offloading method that caches experts for whole segments, under a cache size limit related to the number of active experts. SCH measures local routing consistency on model- and router-levels, yet is more related to the performance of *real expert offloading systems* as it accounts for the cache limit.

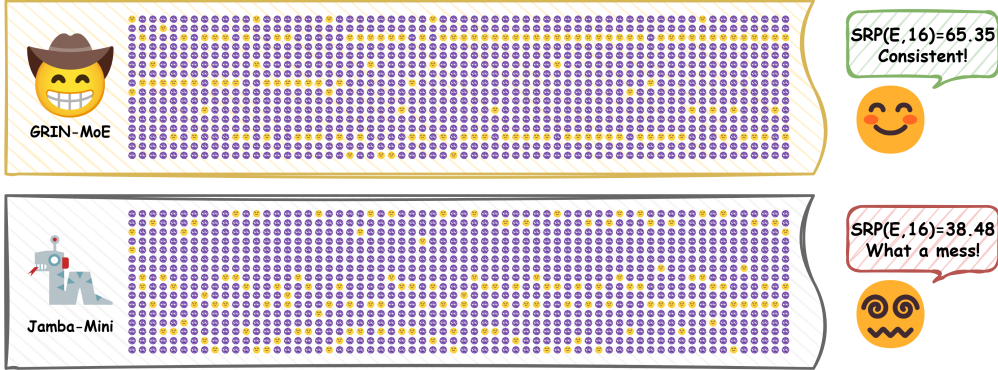


Figure 1: Routing results by GRIN-MoE (Liu et al., 2024b) layer 21 and Jamba-Mini-1.6 (Lenz et al., 2025) layer 25 on the same input (Java code). Despite having similar model sizes and the same number of experts, GRIN-MoE exhibits more consistent routing patterns than Jamba-Mini-1.6, with multiple experts continuously activated, so expert caching with GRIN-MoE will be more feasible and effective.

We conduct experiments on 20 MoE-based LLMs, covering parameter scales ranging from 3 billion to 54 billion and encompassing diverse architectures. While most models exhibit similar local routing consistency within a few tokens, the variance enlarges when the segment length increases. Models that **apply MoE on every layer** and **do not use shared experts** exhibit the **highest local routing consistency**, some of which even achieve load balance. Additionally, we investigate local routing consistency across different context domains and assess its relationship with experts’ domain and vocabulary preferences. The findings reveal that **domain-specialized experts**, if they exist, **contribute more** to local routing consistency, whereas vocabulary specialization has less impact. Finally, we verify the strong correlation between SRP and SCH, confirming that both metrics effectively represent local routing consistency. We conclude that most MoE models can achieve

optimal balance between segment caching effectiveness and deployment efficiency with a **cache size approximately 2x the number of active experts**.*

Overall, our contributions are threefold:

1. We propose *local routing consistency*, a property of MoE models that reflects the potential efficiency of expert offloading for the model. We design two metrics to quantify local routing consistency: segment routing best performance (SRP), which provides parameter-free fine-grained analysis, and segment cache best hit rate (SCH), aligned with practical expert offloading.
2. We conduct empirical analysis across 20 MoE-based LLMs, demonstrating that models that apply MoE on every layer and without shared experts show the highest local routing consistency. We also reveal that domain-specialized experts contribute more to local routing consistency than vocabulary-specialized experts.
3. We analyze SCH under different cache sizes relative to the number of active experts. Alongside the optimal cache size derived from SRP results, we conclude that cache sizes 2x the size of active parameters achieve the best segment caching results on most models.

2 Definitions

2.1 Preliminary: mixture of experts

Transformer-based language models have most parameters on feed-forward network (FFN) layers. As the model size scales up, LLMs often replace them with sparse MoE layers to reduce computational cost during inference. A typical MoE layer with E experts can be parameterized by E smaller FFNs $F_1(\cdot; \theta_1), \dots, F_E(\cdot; \theta_E)$, where each $F_i: \mathbb{R}^d \rightarrow \mathbb{R}^d$ defines a single expert. There is also a router $R: \mathbb{R}^d \rightarrow \mathbb{R}^E$ in the MoE layer to choose experts and give weights. For each token x with hidden representation $h_x \in \mathbb{R}^d$, its output is given by

$$[s_1, \dots, s_E] = \text{Softmax}(R(h_x)); [w_1, \dots, w_E] = \text{Top}_k(s_1, \dots, s_E); o_x = \sum_{i=1}^E w_i F_i(h_x; \theta_i) \quad (1)$$

where Top_k preserves the k largest scores and sets others to 0. Experts with a null score can be effectively deactivated without calculating, thus saving computational resources. Other components in the transformer architecture may also be replaced by their MoE variant, such as Mixture-of-Attention (Zhang et al., 2022) for self-attention and MixLoRA (Li et al., 2024a) for LoRA adapters. Nevertheless, we focus on MoE layers that replace FFNs, as this is the most prominent and effective design (in terms of the number of parameters). More discussions about MoE LLMs can be found in Appendix B, where we also briefly review expert offloading for MoE models.

The above routing procedure is done token by token, which does not guarantee consecutive routing. Since the consistency of consecutive routing decisions can benefit expert offloading systems, it is necessary to investigate the degree of consecutive routing of different MoE-based LLMs. In the following sections, we propose two metrics to measure this **local routing consistency** and compare them across MoE models with different structure parameters.

2.2 Segment routing best performance (SRP)

An intuitive way to measure local routing consistency is to compare the distribution of routing choices between tokens in a continuous segment. However, typical metrics for distribution comparison, such as the Kullback-Leibler divergence, work with two distributions and thus are unsuitable for our demand. Instead, we measure how well a simplified, segment-based router can mimic the behavior of the original token-based router. Here we define the **segment routing best performance (SRP)** of a single expert e and a group of experts E (e.g., experts from the same MoE layer or model).

Single expert For any input sequence $T = [t_1, \dots, t_{|T|}]$, we denote the activation sequence of expert e on T as $A(e, T) = [a_1, \dots, a_{|T|}]$, where $a_i \in \{0, 1\}$ indicates whether e is activated on t_i ($a_i = 1$) or not ($a_i = 0$). A segment-based router R_e^m with segment length $m > 0$ for expert e will

*We propose this insight from the aspect of model design, especially when building models for devices with known memory constraints (e.g., mobile phones).

try to mimic $[a_p, \dots, a_{p+m-1}]$ for any T and p , similar to expert-choice routing (Zhou et al., 2022). Let $R_e^m(T, p) = [b_1, \dots, b_m]$ be its prediction, with the segment-prediction constraint:

$$b_i = 0, \forall i = 1, \dots, m \quad \text{or} \quad b_i = 1, \forall i = 1, \dots, m \quad (2)$$

In other words, R_e^m decides that e is either always active or always inactive on $[t_p, \dots, t_{p+m-1}]$. For simplicity, we write $R_e^m(T, p) = 0$ and $R_e^m(T, p) = 1$ for the two cases respectively. By treating each segment routing attempt as a binary classification task with m samples, and considering all possible segments of all possible inputs, we can calculate the F_1 score of R_e^m :

$$F_1(R_e^m) = \frac{2 \sum_T \sum_{p=1}^{|T|-m+1} R_e^m(T, p) \cdot f(e, T, p, m)}{\sum_T \sum_{p=1}^{|T|-m+1} [m \cdot R_e^m(T, p) + f(e, T, p, m)]} \quad (3)$$

where $f(e, T, p, m) = \sum_{i=p}^{p+m-1} A(e, T)[i]$ is the active frequency of e in the segment of T with length m starting at position p . We demonstrate the detailed process to obtain this equation in Appendix C.1. We choose F_1 score because we do not set up an upper bound on how many experts R_e^m can be selected, so hit rate (recall) can go up to 1 by selecting all experts. Moreover, missing major experts is worse than activating minor ones, as changing low-rank experts usually affects model performance less (Kong et al., 2024; Skliar et al., 2024); therefore, we prefer F_1 score to accuracy to emphasize this difference.

Based on Equation 3, we define the segment routing best performance of e under segment length m as the maximum F_1 score any R_e^m can achieve: $\text{SRP}(e, m) \triangleq \max_{R_e^m} F_1(R_e^m)$. Furthermore, in Appendix C.2 we prove that $F_1(R_e^m)$ is maximized if and only if R_e^m gives active predictions for all segments that activates e at least α_e^m times, where $\alpha_e^m \in [0, m]$ is only related to e and m :

$$\text{SRP}(e, m) = \frac{2 \sum_T \sum_{f(e, T, p, m) \geq \alpha_e^m} f(e, T, p, m)}{\sum_T \sum_{p=1}^{|T|-m+1} [m \cdot I[f(e, T, p, m) \geq \alpha_e^m] + f(e, T, p, m)]} \quad (4)$$

Therefore, $\text{SRP}(e, m)$ is an intrinsic property of the expert that reflects its local routing consistency, unrelated to any specific segment routing methods.

Expert group For a group of experts E , let R_E^m be a segment-based router that decides whether each expert $e \in E$ should be activated in a segment of some input T with length m ; more specifically, $R_E^m(e, T, p)$ is a prediction sequence similar to $R_e^m(T, p)$ that also follows the segment-prediction constraint (Equation 2). Following the same procedure in Appendix C.1, we have

$$F_1(R_E^m) = \frac{2 \sum_T \sum_{p=1}^{|T|-m+1} \sum_{e \in E} R_E^m(e, T, p) \cdot f(e, T, p, m)}{\sum_T \sum_{p=1}^{|T|-m+1} \sum_{e \in E} [m \cdot R_E^m(e, T, p) + f(e, T, p, m)]} \quad (5)$$

Again, $F_1(R_E^m)$ is maximized if and only if R_E^m gives active predictions for all expert-segment pairs where the expert is activated at least α_e^m times in the segment, where α_e^m is decided by E and m . Therefore we have

$$\text{SRP}(E, m) \triangleq \max_{R_E^m} F_1(R_E^m) = \frac{2 \sum_T \sum_{f(e, T, p, m) \geq \alpha_e^m} f(e, T, p, m)}{\sum_T \sum_{p=1}^{|T|-m+1} \sum_{e \in E} [m \cdot I[f(e, T, p, m) \geq \alpha_e^m] + f(e, T, p, m)]} \quad (6)$$

$\text{SRP}(E, m)$ measures how well a group of experts is coordinated by the original router(s) to achieve layer-level or model-level local routing consistency.

Segment routing size ratio Consider the case where the expert group E is the set of all experts from an MoE layer that activate top- k experts. When we use the prediction of R_E^m that achieves the highest F_1 to route experts, for any segment of input T with length m starting at position p , only experts that is activated at least α_e^m times by the original router will be chosen. Hence, the expectation of the number of routed experts for any segment is given by

$$k_m = \sum_T \frac{1}{|T| - m + 1} \sum_{p=1}^{|T|-m+1} \sum_{e \in E} I[f(e, T, p, m) \geq \alpha_e^m] \quad (7)$$

That is, R_E^m will select k_m experts on average. We define the ratio between k_m and k as the **segment routing size ratio**: $\rho(E, m) \triangleq k_m/k$. A small $\rho(E, m)$ indicates that the local routing consistency of the experts is high enough, so that segment routing does not need to select too many experts to cover real demands under average cases. We use it as a supplementary metric to distinguish between cases where groups of experts have similar segment routing best performances. Note that $\rho(E, m)$ is also available for the entire model as long as the total number of activated experts for each token is also fixed.

Note that in the above definitions, T is unbounded, causing $\text{SRP}(e, m)$, $\text{SRP}(E, m)$ and $\rho(E, m)$ to be uncomputable. In experiments, we choose a fixed corpora S (general or domain-specific) and let $T \in S$, in which case we write $\text{SRP}_S(e, m)$, $\text{SRP}_S(E, m)$ and $\rho_S(E, m)$ to indicate the corpora. Furthermore, we only need to consider α_e^m and α_E^m that are integers, thus we can effectively enumerate from 0 to m to find the corresponding α_e^m and α_E^m respectively.

2.3 Segment cache best hit rate (SCH)

The advantage of SRP as a local routing consistency metric is that it only relies on the expert e (or expert group E) and the segment length m , and is suitable to analyze individual experts. However, real expert offloading scenarios usually have a hard cache size limit, so the best global F_1 score may not be achievable. Moreover, when considering caching performance, F_1 score is not as straightforward as hit rate (recall). Therefore, we propose another metric for local routing consistency, namely **segment cache best hit rate (SCH)**, that is more related to expert offloading.

We use the same notations as above, and stick to the expert group case where E denotes a group of experts that activates k experts for each token. Instead of a segment-based router R_E^m , here we consider a segment-based cache $C_E^{m, \rho}$ where m is the segment length, and ρ be the **segment cache size ratio**; in other words, $C_E^{m, \rho}$ has a cache size of ρk . For any input T and any of its sub-segments of length m starting at position p , $C_E^{m, \rho}$ will choose ρk experts to cache throughout the segment, without switching any experts. Let $C_E^{m, \rho}(e, T, p) = 1$ if an expert $e \in E$ is cached for the segment, and $C_E^{m, \rho}(e, T, p) = 0$ otherwise, then the hit rate of $C_E^{m, \rho}$ across all segments of all possible inputs is given by

$$r(C_E^{m, \rho}) = \frac{\sum_T \sum_{p=1}^{|T|-m+1} \sum_{e \in E} C_E^{m, \rho}(e, T, p) \cdot f(e, T, p, m)}{\sum_T \sum_{p=1}^{|T|-m+1} \sum_{e \in E} f(e, T, p, m)} \quad (8)$$

We define the segment cache best hit rate of E under segment length m and segment cache size ratio ρ as the maximum hit rate any $C_E^{m, \rho}$ can reach: $\text{SCH}(E, m, \rho) = \max_{C_E^{m, \rho}} r(C_E^{m, \rho})$. Note that in Equation 8, the denominator is unrelated to $C_E^{m, \rho}$; therefore, to maximize $r(C_E^{m, \rho})$, we only need to make sure that $C_E^{m, \rho}$ always caches the ρk experts with the highest f :

$$\text{SCH}(E, m, \rho) = \frac{\sum_T \sum_{p=1}^{|T|-m+1} \sum \text{Top}_{\rho k} \{f(e, T, p, m) | e \in E\}}{\sum_T \sum_{p=1}^{|T|-m+1} \sum_{e \in E} f(e, T, p, m)} \quad (9)$$

We write $\text{SCH}_S(E, m, \rho)$ when we limit T to a fixed corpora S .

3 SRP-based consistency analysis

3.1 Experiment setup

Models We conduct experiments on 20 MoE-based LLMs with model sizes ranging from 3B to 57B, covering both popular (SwitchTransformers, Mixtral) and recent (DeepSeek-V2, Qwen3) models. We list their architecture and configuration details in Appendix D.1. We may also use shorter names (e.g., OLMoE and Qwen3) when there is no ambiguity.

Given that many models also have post-trained (e.g., SFT, RL) versions, we compared the local routing consistency between base and post-trained versions of several models in Appendix E.1, where we found no significant difference. Therefore, we always choose the base version in our main experiments.

Data We construct our sample corpus S from two sources: (1) **Generic Corpora**: We include all 7 categories from RedPajama (Together Computer, 2023), including C4, CommonCrawl, Books, Wikipedia, ArXiv, StackExchange, and GitHub. (2) **Downstream Application**: We append several datasets with cases aligned with modern LLM applications, including arena-human-preference-140k (LMArena; LMArena, 2025, OpenMathInstruct-2 (OpenMath; Toshniwal et al., 2025), OpenCodeInstruct (OpenCode; Ahmad et al., 2025), and OpenScienceReasoning-2 (OpenScience; NVIDIA Corporation, 2025). We treat each RedPajama category and downstream application dataset as a distinct domain, and refer to its subset corpus using the data source (Books, GitHub, etc.). The full corpus, referred to as S , contains 22,528 input samples, each sample having 512 tokens. Appendix D.2 gives more details on the data processing and input generation process.

Method and configuration We collect every MoE layer’s routing decisions for every input[†], and for each expert, count the number of activated tokens f in every segment. Although modern LLMs utilize position encodings to distinguish tokens from different positions, we demonstrate in Appendix E.4 that segments from different positions have nearly identical SRP, except for the very first ones that may contain special head tokens. Therefore, we perform calculations on all segments and do not care about their positions. To obtain SRP, we count the number of segments with the same f for each expert (group), then compute the F_1 score for every α candidate, choose the α that achieves the highest F_1 and finally obtain the segment routing best performance and size ratio. We run all experiments on 8 NVIDIA GeForce RTX 4090 graphics cards and load model parameters with BF16 precision.

3.2 Overall results

Figure 2 illustrates $\text{SRP}(E, m)$ of each model under various segment lengths m . While most models have similar SRP and ρ when $m = 4$, the difference between models becomes significant as the segment length increases. Meanwhile, the relative distribution of SRP across models appears similar after $m = 16$. There is a gap between short-term ($m = 4$) and long-term ($m \geq 16$) local routing consistency, where **many models exhibit the short-term one but only a few demonstrate the long-term one**.

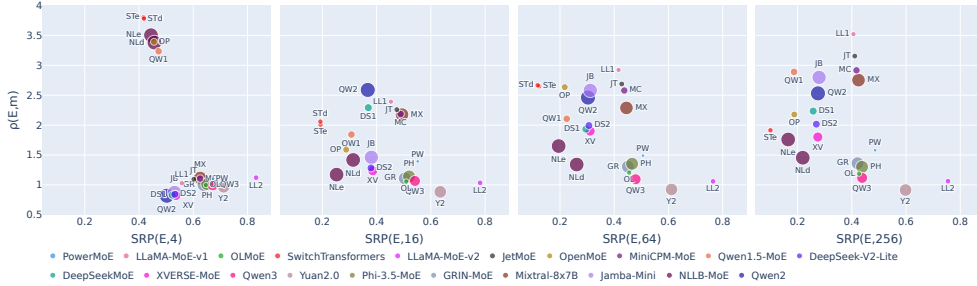


Figure 2: $\text{SRP}(E, m)$ of each model on S , compared with the segment routing size ratio. Marker size represents model size.

We roughly divide the models into four groups that have similar SRP characteristics, whose SRP when $m = 16$ are demonstrated in Table 1:

- Group 1 (LLaMA-MoE-v2-OLMoE) has the highest SRP (> 0.5 when $m = 16$) and ρ (~ 1.25) across all segment lengths, showing strong long-term local routing consistency.
- Group 2 (Mixtral-8x7B-LLaMA-MoE-v1) have the second highest SRP (~ 0.48 when $m = 16$), but their long-term ρ becomes high (~ 2.5).
- Group 3 (XVERSE-MoE-DeepSeekMoE) has a significantly lower SRP than group 2, especially the long-term one (~ 0.36 when $m = 16$), but their ρ is a bit lower ~ 2.0 .
- Group 4 (NLLB-MoE-SwitchTransformers) have the lowest SRP (< 0.31 when $m = 16$); contrasting other models, their short-term ρ is already high, but their long-term ρ becomes lower.

[†]In encoder-decoder models, encoder layers only consider encoder input, and decoder layers likewise.

Table 1: $\text{SRP}(E, 16)$ on S in descending order and several architecture parameters. “A:T”: ratio between active and all experts; “S:A”: ratio between shared and active experts; “every x ”: apply MoE every x layers; “after 1st”: apply MoE after the first layer.

Model	SRP	MoE	A:T	S:A	Model	SRP	MoE	A:T	S:A
LLaMA-MoE-v2	78.16	all	1:4	0	XVERSE-MoE	38.58	all	3:32	1:3
Yuan2.0	63.48	all	1:16	0	Jamba-Mini	38.08	every 2	1:8	0
PowerMoE	55.17	all	1:5	0	DeepSeek-V2-Lite	37.92	after 1st	3:32	1:3
Qwen3	54.14	all	1:16	0	DeepSeekMoE	36.94	after 1st	3:32	1:3
Phi-3.5-MoE	51.98	all	1:8	0	Qwen2	36.74	all	1:8	1:1
OLMoE	50.91	all	1:8	0	NLLB-MoE (encoder)	25.24	every 4	1:64	0
GRIN-MoE	50.39	all	1:8	0	(decoder)	31.35			
Mixtral-8x7B	49.36	all	1:4	0	Qwen1.5-MoE	30.71	all	1:15	1:1
MiniCPM-MoE	48.85	all	1:4	0	OpenMoE	28.77	every 6	1:16	1:2
JetMoE	47.45	all	1:4	0	SwitchTF (encoder)	19.33	every 2	1:128	0
LLaMA-MoE-v1	45.29	all	1:4	0	(decoder)	19.27			

3.3 Analysis

Which model architecture has the highest local routing consistency? We compare the SRP of different models with some architecture parameters in Table 1. Surprisingly, the least sparse models **do not always** have very high local routing consistency, even though they activate experts more frequently on average. Instead, we observe two common points that are possibly connected to high local routing consistency: (1) **Apply MoE on every layer**: all models in groups 1 and 2 apply the MoE structure on every layer, while many group 3 and 4 models skip some layers (e.g., Jamba). (2) **No shared experts**: all models in groups 1 and 2 do not include any shared experts, unlike many group 3 and 4 models (e.g., Qwen2). Both ensure that no dense modules (dense layers or shared experts) are engaged. We conjecture that dense modules may interfere or weaken MoE routing signals, resulting in (a) smoother routing distributions that cause inconsistency over edge cases, and (b) less specialized experts that contribute to local routing consistency (see Section 4). Thus, removing them effectively eliminates these negative factors. We also compare SRP with corpus perplexity (Appendix E.2) and other model designs (Appendix E.3), where we find little correlation.

Table 2: $\text{SRP}(E, 16)$ on S , along with load balance (LB) measured by activation frequency standard deviation of experts.

Model	SRP	LB	Model	SRP	LB	Model	SRP	LB
LLaMA-MoE-v2	78.16	29.04	Mixtral-8x7B	49.36	2.70	DeepSeekMoE	36.94	2.03
Yuan2.0	63.48	13.86	MiniCPM-MoE	48.85	2.59	Qwen2	36.74	6.74
PowerMoE	55.17	12.90	JetMoE	47.45	1.12	NLLB-MoE (en)	25.24	1.75
Qwen3	54.14	3.19	LLaMA-MoE-v1	45.29	2.66	(de)	31.35	2.13
Phi-3.5-MoE	51.98	4.90	XVERSE-MoE	38.58	2.71	Qwen1.5-MoE	30.71	0.58
OLMoE	50.91	6.79	Jamba-Mini	38.08	3.05	OpenMoE	28.77	2.56
GRIN-MoE	50.39	3.89	DeepSeek-V2-Lite	37.92	2.34	SwitchTF (en)	19.33	0.58
						(de)	19.27	0.66

Can local routing consistency live with load balance? Load balance is a key feature for efficient MoE inference (Lepikhin et al., 2021), but it is seemingly against local routing consistency. For instance, both DeepSeek-AI et al. (2024b) and Skliar et al. (2024) suggest adding bias to router outputs. Still, the former promotes *little activated* experts for load balance and the latter promotes *recently cached* experts for effective caching. To investigate their relation, we compute the standard deviation of all experts’ activation frequencies in a model and compare it with SRP in Table 2. Many models with high SRP also exhibit imbalanced routing, which largely contributes to their local routing consistency (details in Appendix E.9). However, models like Qwen3 and GRIN-MoE enjoy high local routing consistency and moderate load balance simultaneously, in which we found strong domain-specialized experts (see Section 4). Based on the observations, we claim that **local routing consistency can coexist with load balance through domain-specialized experts**.

4 Local routing consistency and expert specialization

4.1 Domain-wise local routing consistency

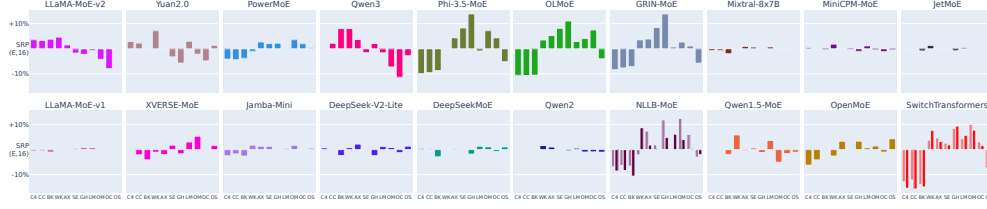


Figure 3: $SRP_D(E, 16)$ on each domain D , relative to $SRP_S(E, 16)$. C4: C4; CC: CommonCrawl; BK: Books; WK: Wikipedia; AX: ArXiv; SE: StackExchange; GH: GitHub; LM: LMArena; OM: OpenMath; OC: OpenCode; OS: OpenScience. For encoder-decoder models, light color represents the encoder and dark color represents the decoder.

In Section 3, we analyze models on the full corpus S , which consists of text data from 7 different domains. However, each domain has its token distributions, which may affect the router decision’s distribution. Figure 3 illustrates the relative difference between domain-wise and global SRP of each model when $m = 16$, where we observe three different patterns among all models: (1) Models like Phi-3.5-MoE, GRIN-MoE, and OLMoE have significantly **higher SRP on ArXiv, StackExchange and GitHub**, whose SRP can be more than 10% higher than global SRP. They also exhibit higher SRP on OpenMath and OpenCode, indicating specialized experts for math and coding tasks. (2) Models like LLaMA-MoE-v2, Yuan2.0, and Qwen3 have significant **higher SRP on Wikipedia and other generic domains** but lower on OpenMath and OpenCode. They seem to have specialized experts for generic text (e.g., multilingual experts) instead of math or coding. (3) Models like Mixtral-8x7B, MiniCPM-MoE and JetMoE have **similar SRP across all domains** with insignificant differences. All of them have mediocre to low SRP. Above all, models exhibit balanced local routing consistency across all domains or higher local routing consistency on certain domains with unique properties.

4.2 Expert specialization

We argue that domain-wise local routing consistency patterns appear across models due to specialized experts in each model. To clarify this, we consider two types of expert specialization, first introduced by Muennighoff et al. (2025): (1) **Domain specialization**: the normalized frequency of an expert e being activated on tokens from a specific domain D . We compute the coefficient of variation (CV) of activation frequency across all domains as a domain-free metric. (2) **Vocabulary specialization**: the normalized frequency of an expert e being activated on a specific token ID x . We follow Muennighoff et al. (2025) to obtain the vocabulary specialization of each expert. We compare each model’s SRP, average expert specialization, and the correlation between experts’ specialization and SRP in Figure 4; expert distribution between specialization and SRP is also demonstrated in Appendix E.9.

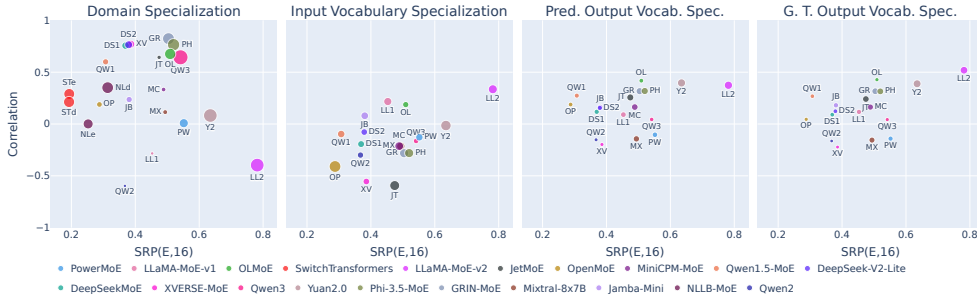


Figure 4: $SRP(E, 16)$ of each model, compared with average expert specialization (marker size) and the correlation between expert specialization and SRP (y-axis).

Domain specialization Many models show a positive correlation between their experts’ domain specialization and SRP. An exception is LLaMA-MoE-v2, which constantly activates a group of experts, resulting in very high SRP. Other exceptions (Qwen2 and LLaMA-MoE) hardly have any domain specialization. In contrast, Qwen3, Phi-3.5-MoE, GRIN-MoE, and OLMoE exhibit high SRP, high average domain specialization, and strong correlation between them simultaneously. These models also demonstrate good load balance (see Table 2), indicating that their **high proportion of domain specialized experts contributes to both local routing consistency and load balance**.

Vocabulary specialization We consider three kinds of vocabulary specialization on the input, the model’s predicted output, and the ground-truth, respectively (Muennighoff et al., 2025). Most models demonstrate negative or insignificant correlation between *input* vocabulary specialization and SRP; LLaMA-MoE-v2 becomes the only exception, also due to the constantly activated experts. On the other hand, SRP is slightly positively correlated to *prediction* or *ground* truth vocabulary specialization. We conjecture that such specialization happens more in later layers (Muennighoff et al., 2025) that process high-level information related to the context topic.

Above all, we can see that **domain specialization plays a more important role in forming local routing consistency than vocabulary specialization**, especially on load-balanced models.

5 SCH-based consistency analysis

5.1 Overall results

As mentioned in Section 2.3, SRP has several flaws that hinder its application in expert offloading. This section focuses on the segment cache best hit rate (SCH), which works with a size limit, to obtain a more straightforward insight into expert offloading and cache management. Based on the number of activated tokens in every input segment from the corpus for each expert, defined as f in Section 3.1, we calculate each model and each layer’s SCH on every possible cache size: At each segment, we sort experts in each layer or model by their f in descending order, and compute the segment cache best hit rate at cache sizes ranging from 1 to $|E|$.

Figure 5 illustrates $SCH(E, m, \rho)$ of each model under different m s and ρ s. We can easily identify the four groups of models mentioned in Section 3.2 starting from $m = 16$: Group 1 models have the fastest growing SCH with respect to ρ when ρ is small, as well as turning points near $\rho = 2$, after which they share similar SCH with group 2 models. Meanwhile, models from groups 3 and 4 have relatively low SCH, growing nearly linearly as ρ increases. Since only group 1 models (with the highest local routing consistency) have turning points on SCH, we claim that in general, $\rho = 2$ can **balance cache effectiveness and efficiency**.

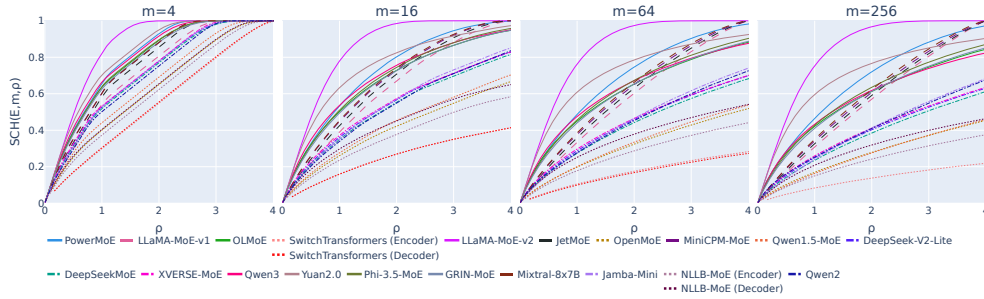


Figure 5: $SCH(E, m, \rho)$ of each model on S under different segment length m and segment cache size ratio ρ . Solid line: group 1; dashed: group 2; dash-dotted: group 3; dotted: group 4.

5.2 SCH vs. common cache algorithm hit rate

Since SCH is based on an ideal cache system that relies on oracle information, it is crucial to understand how well it is correlated with real-world implementations. Table 3 lists the correlation between SCH and the cache hit rate of several widely adopted cache algorithms. All compared cache algorithms have hit rates highly correlated to SCH, even for short segments ($m = 4$). Furthermore,

Table 3: Correlation between SCH and the hit rate of common cache algorithms across all models. LRU: least recently used; LFU: least frequently used; Fixed: fixed cache.

m	LRU	LFU	Fixed
4	81.20	77.39	76.26
16	90.43	88.70	89.79
64	93.10	92.82	95.50
256	97.52	99.20	97.91

Appendix E.6 reveals that SRP and SCH are also highly correlated with each other. This suggests that models with higher local routing consistency tend to achieve higher expert cache hit rates, hence greater performance gain with expert offloading.

6 Conclusion

In this paper, we investigate the property of MoE LLMs where similar experts can be continuously activated, namely *local routing consistency*. We propose two metrics to measure this property: segment routing best performance (SRP) and segment cache best hit rate (SCH). We compare SRP and SCH between multiple models and identify several key designs that may help improve local routing consistency of MoE LLMs. We further suggest that a cache size approximately 2x the number of active experts can balance cache effectiveness and efficiency.

Ethics statement Our analytic methods and results still help design new MoE LLMs friendly to expert offloading and enable deployment on resource-constrained edge devices. While our study will likely have an indirect social impact, developers implementing local routing consistency to build more powerful LLMs must take responsibility for their products’ societal implications.

Reproducibility statement We constructed our sample corpus S with deterministic algorithms, and we will publish the sampled S to ensure reproducibility. We also conducted all experiments with a deterministic configuration and will release relevant source code.

References

Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, Alon Benhaim, Misha Bilenko, Johan Bjorck, Sébastien Bubeck, Martin Cai, Qin Cai, Vishrav Chaudhary, Dong Chen, Dongdong Chen, Weizhu Chen, Yen-Chun Chen, Yi-Ling Chen, Hao Cheng, Parul Chopra, Xiyang Dai, Matthew Dixon, Ronen Eldan, Victor Fragoso, Jianfeng Gao, Mei Gao, Min Gao, Amit Garg, Allie Del Giorno, Abhishek Goswami, Suriya Gunasekar, Emman Haider, Junheng Hao, Russell J. Hewett, Wenxiang Hu, Jamie Huynh, Dan Iter, Sam Ade Jacobs, Mojan Javaheripi, Xin Jin, Nikos Karampatziakis, Piero Kauffmann, Mahoud Khademi, Dongwoo Kim, Young Jin Kim, Lev Kurilenko, James R. Lee, Yin Tat Lee, Yuanzhi Li, Yunsheng Li, Chen Liang, Lars Liden, Xihui Lin, Zeqi Lin, Ce Liu, Liyuan Liu, Mengchen Liu, Weishung Liu, Xiaodong Liu, Chong Luo, Piyush Madan, Ali Mahmoudzadeh, David Majercak, Matt Mazzola, Caio César Teodoro Mendes, Arindam Mitra, Hardik Modi, Anh Nguyen, Brandon Norick, Barun Patra, Daniel Perez-Becker, Thomas Portet, Reid Pryzant, Heyang Qin, Marko Radmilac, Liliang Ren, Gustavo de Rosa, Corby Rosset, Sambudha Roy, Olatunji Ruwase, Olli Saarikivi, Amin Saied, Adil Salim, Michael Santacroce, Shital Shah, Ning Shang, Hiteshi Sharma, Yelong Shen, Swadheen Shukla, Xia Song, Masahiro Tanaka, Andrea Tupini, Praneetha Vaddamanu, Chunyu Wang, Guanhua Wang, Lijuan Wang, Shuohang Wang, Xin Wang, Yu Wang, Rachel Ward, Wen Wen, Philipp Witte, Haiping Wu, Xiaoxia Wu, Michael Wyatt, Bin Xiao, Can Xu, Jiahang Xu, Weijian Xu, Jilong Xue, Sonali Yadav, Fan Yang, Jianwei Yang, Yifan Yang, Ziyi Yang, Donghan Yu, Lu Yuan, Chenruidong Zhang, Cyril Zhang, Jianwen Zhang, Li Lyna Zhang, Yi Zhang, Yue Zhang, Yunan Zhang, and Xiren Zhou. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint*, April 2024. doi: 10.48550/ARXIV.2404.14219.

- Wasi Uddin Ahmad, Aleksander Ficek, Mehrzad Samadi, Jocelyn Huang, Vahid Noroozi, Somshubra Majumdar, and Boris Ginsburg. Opencodestruct: A large-scale instruction tuning dataset for code llms. *arXiv preprint*, April 2025. doi: 10.48550/ARXIV.2504.04030.
- Weilin Cai, Juyong Jiang, Fan Wang, Jing Tang, Sunghun Kim, and Jiayi Huang. A survey on mixture of experts in large language models. *IEEE Transactions on Knowledge and Data Engineering (TKDE) 2025*, pp. 1–20, June 2024. ISSN 2326-3865. doi: 10.1109/tkde.2025.3554028.
- Shengzhuang Chen, Ying Wei, and Jonathan Richard Schwarz. Automatic expert discovery in llm upcycling via sparse interpolated mixture-of-experts. *arXiv preprint*, June 2025. doi: 10.48550/ARXIV.2506.12597.
- Tianyu Chen, Shaohan Huang, Yuan Xie, Binxing Jiao, Daxin Jiang, Haoyi Zhou, Jianxin Li, and Furu Wei. Task-specific expert pruning for sparse mixture-of-experts. *arXiv preprint*, June 2022. doi: 10.48550/ARXIV.2206.00277.
- Damai Dai, Chengqi Deng, Chenggang Zhao, R.x. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y.k. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1280–1297, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.70. URL <https://aclanthology.org/2024.acl-long.70/>.
- DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Yang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jin Chen, Jingyang Yuan, Junjie Qiu, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruizhe Pan, Runxin Xu, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Size Zheng, T. Wang, Tian Pei, Tian Yuan, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Liu, Xin Xie, Xingkai Yu, Xinnan Song, Xinyi Zhou, Xinyu Yang, Xuan Lu, Xuecheng Su, Y. Wu, Y. K. Li, Y. X. Wei, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Zheng, Yichao Zhang, Yiliang Xiong, Yilong Zhao, Ying He, Ying Tang, Yishi Piao, Yixin Dong, Yixuan Tan, Yiyuan Liu, Yongji Wang, Yongqiang Guo, Yuchen Zhu, Yudian Wang, Yuheng Zou, Yukun Zha, Yunxian Ma, Yuting Yan, Yuxiang You, Yuxuan Liu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhewen Hao, Zhihong Shao, Zhiniu Wen, Zhipeng Xu, Zhongyu Zhang, Zhuoshu Li, Zihan Wang, Zihui Gu, Zilin Li, and Ziwei Xie. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint*, May 2024a. doi: 10.48550/ARXIV.2405.04434.
- DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shutong Pan, T. Wang, Tao

- Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. Deepseek-v3 technical report. *arXiv preprint*, December 2024b. doi: 10.48550/ARXIV.2412.19437.
- Zhixu Du, Shiyu Li, Yuhao Wu, Xiangyu Jiang, Jingwei Sun, Qilin Zheng, Yongkai Wu, Ang Li, Hai Helen Li, and Yiran Chen. Sida: Sparsity-inspired data-aware serving for efficient and scalable large mixture-of-experts models. In P. Gibbons, G. Pekhimenko, and C. De Sa (eds.), *Proceedings of Machine Learning and Systems*, volume 6, pp. 224–238, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/file/698cfaf72a208aef2e78bcac55b74328-Paper-Conference.pdf.
- Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint*, December 2023. doi: 10.48550/ARXIV.2312.17238.
- Zhiyuan Fang, Zicong Hong, Yuegui Huang, Yufeng Lyu, Wuhui Chen, Yue Yu, Fan Yu, and Zibin Zheng. Accurate expert predictions in moe inference via cross-layer gate. *arXiv preprint*, February 2025. doi: 10.48550/ARXIV.2502.12224.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL <http://jmlr.org/papers/v23/21-0998.html>.
- Wenfeng Feng, Chuzhan Hao, Yuewei Zhang, Yu Han, and Hao Wang. Mixture-of-LoRAs: An efficient multitask tuning method for large language models. In Nicoletta Calzolari, Min-Yen Kan, Veronique Hoste, Alessandro Lenci, Sakriani Sakti, and Nianwen Xue (eds.), *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pp. 11371–11380, Torino, Italia, May 2024. ELRA and ICCL. URL <https://aclanthology.org/2024.lrec-main.994/>.
- Hongcan Guo, Haolang Lu, Guoshun Nan, Bolun Chu, Jialin Zhuang, Yuan Yang, Wenhao Che, Sicong Leng, Qimei Cui, and Xudong Jiang. Advancing expert specialization for better moe. *arXiv preprint*, May 2025. doi: 10.48550/ARXIV.2505.22323.
- Xin He, Shunkang Zhang, Yuxin Wang, Haiyan Yin, Zihao Zeng, Shaohuai Shi, Zhenheng Tang, Xiaowen Chu, Ivor Tsang, and Ong Yew Soon. Expertflow: Optimized expert activation and token allocation for efficient mixture-of-experts inference. *arXiv preprint*, October 2024. doi: 10.48550/ARXIV.2410.17954.
- Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, Xinrong Zhang, Zheng Leng Thai, Kaihuo Zhang, Chongyi Wang, Yuan Yao, Chenyang Zhao, Jie Zhou, Jie Cai, Zhongwu Zhai, Ning Ding, Chao Jia, Guoyang Zeng, Dahai Li, Zhiyuan Liu, and Maosong Sun. Minicpm: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint*, April 2024. doi: 10.48550/ARXIV.2404.06395.
- Haiyang Huang, Newsha Ardalani, Anna Sun, Liu Ke, Hsien-Hsin S. Lee, Anjali Sridhar, Shruti Bhosale, Carole-Jean Wu, and Benjamin Lee. Towards moe deployment: Mitigating inefficiencies in mixture-of-expert (moe) inference. *arXiv preprint*, March 2023. doi: 10.48550/ARXIV.2303.06182.

- Wei Huang, Yue Liao, Jianhui Liu, Ruifei He, Haoru Tan, Shiming Zhang, Hongsheng Li, Si Liu, and Xiaojuan Qi. Mixture compressor for mixture-of-experts llms gains more. *arXiv preprint*, October 2024. doi: 10.48550/ARXIV.2410.06270.
- Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pp. 1018–1031, June 2024. doi: 10.1109/ISCA59077.2024.00078.
- Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, L  lio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Th  ophile Gerv  t, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mixtral of experts. *arXiv preprint*, January 2024. doi: 10.48550/ARXIV.2401.04088.
- Keisuke Kamahori, Yile Gu, Kan Zhu, and Baris Kasikci. Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models. In *5th Workshop on practical ML for limited/low resource settings*, 2024. URL <https://openreview.net/forum?id=WX7lxohjFe>.
- Rui Kong, Yuanchun Li, Qingtian Feng, Weijun Wang, Xiaozhou Ye, Ye Ouyang, Linghe Kong, and Yunxin Liu. SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6710–6720, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.363. URL <https://aclanthology.org/2024.acl-long.363/>.
- Barak Lenz, Opher Lieber, Alan Arazi, Amir Bergman, Avshalom Manevich, Barak Peleg, Ben Aviram, Chen Almagor, Clara Fridman, Dan Padnos, Daniel Gissin, Daniel Jannai, Dor Muhlgay, Dor Zimberg, Edden M. Gerber, Elad Dolev, Eran Krakovsky, Erez Safahi, Erez Schwartz, Gal Cohen, Gal Shachaf, Haim Rozenblum, Hofit Bata, Ido Blass, Inbal Magar, Itay Dalmedigos, Jhonathan Osin, Julie Fadlon, Maria Rozman, Matan Danos, Michael Gokhman, Mor Zusman, Naama Gidron, Nir Ratner, Noam Gat, Noam Rozen, Oded Fried, Ohad Leshno, Omer Antverg, Omri Abend, Or Dagan, Orit Cohavi, Raz Alon, Ro’i Belson, Roi Cohen, Rom Gilad, Roman Glozman, Shahar Lev, Shai Shalev-Shwartz, Shaked Haim Meirom, Tal Delbari, Tal Ness, Tomer Asida, Tom Ben Gal, Tom Braude, Uriya Pumerantz, Josh Cohen, Yonatan Belinkov, Yuval Globerson, Yuval Peleg Levy, and Yoav Shoham. Jamba: Hybrid transformer-mamba language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=JFPaD7lpBD>.
- Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=qrwe7XHTmYb>.
- Dengchun Li, Yingzi Ma, Naizheng Wang, Zhengmao Ye, Zhiyuan Cheng, Yinghao Tang, Yan Zhang, Lei Duan, Jie Zuo, Cal Yang, and Mingjie Tang. Mixlorax: Enhancing large language models fine-tuning with lora-based mixture of experts. *arXiv preprint*, April 2024a. doi: 10.48550/ARXIV.2404.15159.
- Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. Accelerating distributed MoE training and inference with lina. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pp. 945–959, Boston, MA, July 2023. USENIX Association. ISBN 978-1-939133-35-9. URL <https://www.usenix.org/conference/atc23/presentation/li-jiamin>.
- Pingzhi Li, Zhenyu Zhang, Prateek Yadav, Yi-Lin Sung, Yu Cheng, Mohit Bansal, and Tianlong Chen. Merge, then compress: Demystify efficient SMoe with hints from its routing policy. In *The Twelfth International Conference on Learning Representations*, 2024b. URL <https://openreview.net/forum?id=eFWG9Cy3WK>.
- Jiacheng Liu, Peng Tang, Wenfeng Wang, Yuhang Ren, Xiaofeng Hou, Pheng-Ann Heng, Minyi Guo, and Chao Li. A survey on inference optimization techniques for mixture of experts models. *arXiv preprint*, December 2024a. doi: 10.48550/ARXIV.2412.14219.

- Liyuan Liu, Young Jin Kim, Shuohang Wang, Chen Liang, Yelong Shen, Hao Cheng, Xiaodong Liu, Masahiro Tanaka, Xiaoxia Wu, Wenxiang Hu, Vishrav Chaudhary, Zeqi Lin, Chenruidong Zhang, Jilong Xue, Hany Awadalla, Jianfeng Gao, and Weizhu Chen. Grin: Gradient-informed moe. *arXiv preprint*, September 2024b. doi: 10.48550/ARXIV.2409.12136.
- LMarena. arena-human-preference-140k, August 2025. URL <https://huggingface.co/datasets/lmarena-ai/arena-human-preference-140k>.
- Ka Man Lo, Zeyu Huang, Zihan Qiu, Zili Wang, and Jie Fu. A closer look into mixture-of-experts in large language models. *arXiv preprint*, June 2024. doi: 10.48550/ARXIV.2406.18219.
- Xudong Lu, Qi Liu, Yuhui Xu, Aojun Zhou, Siyuan Huang, Bo Zhang, Junchi Yan, and Hongsheng Li. Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6159–6172, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.334. URL <https://aclanthology.org/2024.acl-long.334/>.
- Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Evan Pete Walsh, Oyvind Tafjord, Nathan Lambert, Yuling Gu, Shane Arora, Akshita Bhagia, Dustin Schwenk, David Wadden, Alexander Wettig, Binyuan Hui, Tim Dettmers, Douwe Kiela, Ali Farhadi, Noah A. Smith, Pang Wei Koh, Amanpreet Singh, and Hannaneh Hajishirzi. OLMoe: Open mixture-of-experts language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=xXTkbTBmqq>.
- Mohammed Muqeeth, Haokun Liu, and Colin Raffel. Soft merging of experts with adaptive routing. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=7I1991c54z>. Featured Certification.
- NLLB Team, Marta R. Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, Anna Sun, Skyler Wang, Guillaume Wenzek, Al Youngblood, Bapi Akula, Loic Barrault, Gabriel Mejia Gonzalez, Prangthip Hansanti, John Hoffman, Semarley Jarrett, Kaushik Ram Sadagopan, Dirk Rowe, Shannon Spruit, Chau Tran, Pierre Andrews, Necip Fazil Ayan, Shruti Bhosale, Sergey Edunov, Angela Fan, Cynthia Gao, Vedanuj Goswami, Francisco Guzmán, Philipp Koehn, Alexandre Mourachko, Christophe Ropers, Safiyyah Saleem, Holger Schwenk, and Jeff Wang. No language left behind: Scaling human-centered machine translation. *arXiv preprint*, July 2022. doi: 10.48550/ARXIV.2207.04672.
- NVIDIA Corporation. OpenScienceReasoning-2, June 2025. URL <https://huggingface.co/datasets/nvidia/OpenScienceReasoning-2>.
- Quang Pham, Giang Do, Huy Nguyen, TrungTin Nguyen, Chenghao Liu, Mina Sartipi, Binh T. Nguyen, Savitha Ramasamy, Xiaoli Li, Steven Hoi, and Nhat Ho. Competesmoe – effective training of sparse mixture of experts via competition. *arXiv preprint*, February 2024. doi: 10.48550/ARXIV.2402.02526.
- Xiaoye Qu, Daize Dong, Xuyang Hu, Tong Zhu, Weigao Sun, and Yu Cheng. Llama-moe v2: Exploring sparsity of llama from perspective of mixture-of-experts with post-training. *arXiv preprint*, November 2024. doi: 10.48550/ARXIV.2411.15708.
- Qwen Team. Qwen1.5-moe: Matching 7b model performance with 1/3 activated parameters", February 2024. URL <https://qwenlm.github.io/blog/qwen-moe/>.
- Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato (eds.), *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 18332–18346. PMLR, July 2022. URL <https://proceedings.mlr.press/v162/rajbhandari22a.html>.

- Jie Ren, Dong Xu, Shuangyan Yang, Jiacheng Zhao, Zhicheng Li, Christian Navasca, Chenxi Wang, Harry Xu, and Dong Li. Enabling large dynamic neural network training with learning-based memory management. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 788–802, March 2024. doi: 10.1109/HPCA57654.2024.00066.
- Noam Shazeer, *Azalia Mirhoseini, *Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=B1ckMDqlg>.
- Sheng Shen, Le Hou, Yanqi Zhou, Nan Du, Shayne Longpre, Jason Wei, Hyung Won Chung, Barret Zoph, William Fedus, Xinyun Chen, Tu Vu, Yuexin Wu, Wuyang Chen, Albert Webson, Yunxuan Li, Vincent Y. Zhao, Hongkun Yu, Kurt Keutzer, Trevor Darrell, and Denny Zhou. Mixture-of-experts meets instruction tuning: A winning combination for large language models. In *The Twelfth International Conference on Learning Representations*, 2024a. URL <https://openreview.net/forum?id=6mLjDwYte5>.
- Yikang Shen, Zheyu Zhang, Tianyou Cao, Shawn Tan, Zhenfang Chen, and Chuang Gan. Moduleformer: Modularity emerges from mixture-of-experts. *arXiv preprint*, June 2023. doi: 10.48550/ARXIV.2306.04640.
- Yikang Shen, Zhen Guo, Tianle Cai, and Zengyi Qin. Jetmoe: Reaching llama2 performance with 0.1m dollars. *arXiv preprint*, April 2024b. doi: 10.48550/ARXIV.2404.07413.
- Yikang Shen, Matthew Stallone, Mayank Mishra, Gaoyuan Zhang, Shawn Tan, Aditya Prasad, Adriana Meza Soria, David D. Cox, and Rameswar Panda. Power scheduler: A batch size and token number agnostic learning rate scheduler. *arXiv preprint*, August 2024c. doi: 10.48550/ARXIV.2408.13359.
- Andrii Skliar, Ties van Rozendaal, Romain Lepert, Todor Boinovski, Mart van Baalen, Markus Nagel, Paul Whatmough, and Babak Ehteshami Bejnordi. Mixture of cache-conditional experts for efficient mobile device inference. *arXiv preprint*, November 2024. doi: 10.48550/ARXIV.2412.00099.
- Xiaoni Song, Zihang Zhong, Rong Chen, and Haibo Chen. Promoe: Fast moe-based llm serving using proactive caching. *arXiv preprint*, October 2024. doi: 10.48550/ARXIV.2410.22134.
- Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. Hobbit: A mixed precision expert offloading system for fast moe inference. *arXiv preprint*, November 2024. doi: 10.48550/ARXIV.2411.01433.
- Together Computer. Redpajama: An open source recipe to reproduce llama training dataset, April 2023. URL <https://github.com/togethercomputer/RedPajama-Data>.
- Shubham Toshniwal, Wei Du, Ivan Moshkov, Branislav Kisacanin, Alexan Ayrapetyan, and Igor Gitman. Openmathinstruct-2: Accelerating AI for math with massive open-source instruction data. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=mTCbq2QssD>.
- Shaohua Wu, Jiangang Luo, Xi Chen, Lingjun Li, Xudong Zhao, Tong Yu, Chao Wang, Yue Wang, Fei Wang, Weixu Qiao, Houbo He, Zeru Zhang, Zeyu Sun, Junxiong Mao, and Chong Shen. Yuan 2.0-m32: Mixture of experts with attention router. *arXiv preprint*, May 2024. doi: 10.48550/ARXIV.2405.17976.
- Fuzhao Xue, Zian Zheng, Yao Fu, Jinjie Ni, Zangwei Zheng, Wangchunshu Zhou, and Yang You. Openmoe: An early effort on open mixture-of-experts language models. In *Forty-first International Conference on Machine Learning*, 2024a. URL <https://openreview.net/forum?id=1YDeZU8Lt5>.
- Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. Moe-infinity: Efficient moe inference on personal machines with sparsity-aware expert cache. *arXiv preprint*, January 2024b. doi: 10.48550/ARXIV.2401.14361.

XVERSE Technology Inc. XVERSE-MoE-A4.2B, April 2024. URL <https://huggingface.co/xverse/XVERSE-MoE-A4.2B>.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yeqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. Qwen2 technical report. *arXiv preprint*, July 2024a. doi: 10.48550/ARXIV.2407.10671.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yeqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint*, May 2025. doi: 10.48550/ARXIV.2505.09388.

Cheng Yang, Yang Sui, Jinqi Xiao, Lingyi Huang, Yu Gong, Yuanlin Duan, Wenqi Jia, Miao Yin, Yu Cheng, and Bo Yuan. MoE-i²: Compressing mixture of experts models through inter-expert pruning and intra-expert low-rank decomposition. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 10456–10466, Miami, Florida, USA, November 2024b. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.612. URL <https://aclanthology.org/2024.findings-emnlp.612/>.

Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. Edgemoe: Empowering sparse large language models on mobile devices. *IEEE Transactions on Mobile Computing*, pp. 1–16, 2025. ISSN 1558-0660. doi: 10.1109/TMC.2025.3546466.

Hanfei Yu, Xingqi Cui, Hong Zhang, Hao Wang, and Hao Wang. fmoe: Fine-grained expert offloading for large mixture-of-experts serving. *arXiv preprint*, February 2025. doi: 10.48550/ARXIV.2502.05370.

Xiaofeng Zhang, Yikang Shen, Zeyu Huang, Jie Zhou, Wenge Rong, and Zhang Xiong. Mixture of attention heads: Selecting attention heads per token. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (eds.), *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 4150–4162, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.278. URL <https://aclanthology.org/2022.emnlp-main.278/>.

Yujie Zhang, Shivam Aggarwal, and Tulika Mitra. Daop: Data-aware offloading and predictive pre-calculation for efficient moe inference. *arXiv preprint*, January 2025. doi: 10.48550/ARXIV.2501.10375.

Shuzhang Zhong, Ling Liang, Yuan Wang, Runsheng Wang, Ru Huang, and Meng Li. Adapmoe: Adaptive sensitivity-based expert gating and management for efficient moe inference. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design, ICCAD '24*, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400710773. doi: 10.1145/3676536.3676741. URL <https://doi.org/10.1145/3676536.3676741>.

Zexuan Zhong, Mengzhou Xia, Danqi Chen, and Mike Lewis. Lory: Fully differentiable mixture-of-experts for autoregressive language model pre-training. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=LKEJPySnlt>.

Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Zhifeng Chen, Quoc V. Le, and James Laudon. Mixture-of-experts with expert choice routing. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 7103–7114. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/2f00ecd787b432c1d36f3de9800728eb-Paper-Conference.pdf.

Tong Zhu, Xiaoye Qu, Daize Dong, Jiacheng Ruan, Jingqi Tong, Conghui He, and Yu Cheng. LLaMA-MoE: Building mixture-of-experts from LLaMA with continual pre-training. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 15913–15923, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.890. URL <https://aclanthology.org/2024.emnlp-main.890/>.

A Use of LLMs

We primarily employ LLMs to polish the writing of this paper, utilizing tools such as Grammarly and Writeful. In all other cases, LLMs are the object of our experiments and analyses, and we do not use them for other purposes.

B Related work

B.1 MoE-based LLM and expert analysis

Since its introduction into large neural networks, MoE has become a critical strategy to build large language models up to trillions of parameters (Shazeer et al., 2017; Fedus et al., 2022; Rajbhandari et al., 2022). While some early models like SwitchTransformers (Fedus et al., 2022) and NLLB (NLLB Team et al., 2022) employ encoder-decoder structures as their backbone, due to the success of GPT-3, the most recent popular MoE-based LLMs use decoder-only structures (Jiang et al., 2024; Yang et al., 2024a; DeepSeek-AI et al., 2024b; Abdin et al., 2024), replacing their original FFN layers with MoE layers containing multiple experts (other components may be replaced too, e.g., self-attention (Shen et al., 2023, 2024b) and LoRA (Li et al., 2024a; Feng et al., 2024)). Cai et al. (2024) systematically introduces MoE architectures and implementation in LLMs.

The popularity of MoE LLMs has triggered interest in understanding how experts are activated in such models. Many model reports and individual studies focused on the relation between expert selection and the input context. For example, Muennighoff et al. (2025) reported that OLMoE shows a significant difference in expert activity across different domains. Contrastively, Xue et al. (2024a) found that the routing choice of OpenMoE is highly related to the input token rather than the input context. Other works investigated the similarity among expert activation patterns (Li et al., 2024b; Lu et al., 2024), as well as the relation between expert output and routing choice (Pham et al., 2024; Lo et al., 2024). Some further proposed methods to reinforce such patterns (Guo et al., 2025; Chen et al., 2025). However, few of them have focused on the local activation pattern of experts. For example, Jiang et al. (2024) reported that in Mixtral, experts are more likely to be activated consecutively, compared to the random case. While their results provide fundamental support for many efficient MoE inference systems Liu et al. (2024a), they only examined the case of 2 consecutive tokens, which may be insufficient to ensure the consistency of expert activation in longer segments.

B.2 Efficient MoE inference and expert offloading

The discrete nature of routers and redundant parameters has caused MoE models to infer more slowly and consume more memory than dense models with the same number of activated parameters. Many techniques have been proposed to boost the inference of MoE models, ranging from model modifications like model compression (Chen et al., 2022; Huang et al., 2024; Yang et al., 2024b; Rajbhandari et al., 2022) and soft routing (Muqeeth et al., 2024; Zhong et al., 2024) to system implementations like load-balanced expert parallel (Lepikhin et al., 2021; Huang et al., 2023; Li et al., 2023) and hardware adaptation (DeepSeek-AI et al., 2024b; Yi et al., 2025). Liu et al. (2024a) provides an in-depth summary of various inference optimization strategies of MoE models.

In this paper, we mainly focus on the potential performance of expert offloading, which enables lossless inference of MoE models on memory-constrained devices by caching only some experts on (fast) memory while leaving others on slow memory or disk storage. Many such systems use pretrained external models and/or information from previous layers to prefetch experts for upcoming layers (Ren et al., 2024; Du et al., 2024; He et al., 2024; Song et al., 2024). Various expert offloading systems propose curated heuristics to manage the expert cache (Skliar et al., 2024; Xue et al., 2024b; Yu et al., 2025; Fang et al., 2025). Among them, some examine the locality of expert activations as empirical support for expert caching efficiency:

- Jiang et al. (2024) first observed that Mixtral-8x7B is likely to choose the same expert for the next token, with probabilities higher than the random expectation. Eliseev & Mazur (2023) extended the argument to 2-4 consecutive tokens through a case study, and boosted inference performance of the same model with LRU caching (plus other techniques such as prefetching and quantization). These works align with our settings, but their analyses are limited to a single model (Mixtral-8x7B), short token spans, and lack a systematic and quantitative study.
- Xue et al. (2024b) reported frequent expert reuse during decoding, and observed that the reused experts depend on the prefilled input. Zhang et al. (2025) found similar routing choices between prefilling and decoding stages. Both observations are utilized to back the effectiveness of expert caching, yet are too coarse in terms of locality (at the input level instead of the token span level).

As more MoE LLMs emerge, understanding what models are more friendly to expert offloading becomes important for the development of both MoE architectures and expert offloading methods.

C Proofs

C.1 Proof of equation 3

In Section 2.2, we consider each routing decision of R_e^m for a segment of length m as a binary classification task with m samples. If we merge all samples from all segments of all possible inputs into one global binary classification task, and define $f(e, T, p, m) = \sum_{i=p}^{p+m-1} A(e, T)[i]$ as in Section 2.2, we will have the following prediction statistics:

$$\begin{aligned} \text{TP}(R_e^m) &= \sum_T \sum_{p=1}^{|T|-m+1} \sum_{i=1}^m I[A(e, T)[p+i-1] = 1 \wedge R_e^m(T, p)[i] = 1] \\ &= \sum_T \sum_{p=1}^{|T|-m+1} R_e^m(T, p) \cdot f(e, T, p, m) \end{aligned} \quad (10)$$

$$\begin{aligned} \text{FP}(R_e^m) &= \sum_T \sum_{p=1}^{|T|-m+1} \sum_{i=1}^m I[A(e, T)[p+i-1] = 0 \wedge R_e^m(T, p)[i] = 1] \\ &= \sum_T \sum_{p=1}^{|T|-m+1} R_e^m(T, p)[m - f(e, T, p, m)] \end{aligned} \quad (11)$$

$$\begin{aligned} \text{FN}(R_e^m) &= \sum_T \sum_{p=1}^{|T|-m+1} \sum_{i=1}^m I[A(e, T)[p+i-1] = 1 \wedge R_e^m(T, p)[i] = 0] \\ &= \sum_T \sum_{p=1}^{|T|-m+1} [1 - R_e^m(T, p)] f(e, T, p, m) \end{aligned} \quad (12)$$

Therefore we have

$$\begin{aligned}
F_1(R_e^m) &= \frac{1}{1/\text{Precision}(R_e^m) + 1/\text{Recall}(R_e^m)} \\
&= \frac{1}{\frac{1}{[TP(R_e^m) + FP(R_e^m)]/TP(R_e^m) + [TP(R_e^m) + FN(R_e^m)]/TP(R_e^m)}} \\
&= \frac{2TP(R_e^m)}{[TP(R_e^m) + FP(R_e^m)] + [TP(R_e^m) + FN(R_e^m)]} \\
&= \frac{2 \sum_T \sum_{p=1}^{|T|-m+1} R_e^m(T, p) \cdot f(e, T, p, m)}{\left[\sum_T \sum_{p=1}^{|T|-m+1} m \cdot R_e^m(T, p) \right] + \left[\sum_T \sum_{p=1}^{|T|-m+1} f(e, T, p, m) \right]} \\
&= \frac{2 \sum_T \sum_{p=1}^{|T|-m+1} R_e^m(T, p) \cdot f(e, T, p, m)}{\sum_T \sum_{p=1}^{|T|-m+1} [m \cdot R_e^m(T, p) + f(e, T, p, m)]}
\end{aligned} \tag{13}$$

which gives Equation 3. $\square$

C.2 Proof of equation 4

Assume that $R_e^m(T_0, p_0) = 0$ for some $e, m > 0, R_e^m, T_0$ and p_0 , then we have

$$\begin{aligned}
F_1(R_e^m) &= \frac{2 \sum_T \sum_{p=1}^{|T|-m+1} R_e^m(T, p) \cdot f(e, T, p, m)}{\sum_T \sum_{p=1}^{|T|-m+1} [m \cdot R_e^m(T, p) + f(e, T, p, m)]} \\
&= \frac{2 \sum_T \sum_{p=1}^{|T|-m+1} R_e^m(T, p) \cdot f(e, T, p, m)}{m \sum_T \sum_{p=1}^{|T|-m+1} R_e^m(T, p) + \sum_T \sum_{p=1}^{|T|-m+1} f(e, T, p, m)} \\
&= \frac{2 \sum_{T \neq T_0 \wedge p \neq p_0} R_e^m(T, p) \cdot f(e, T, p, m)}{m \sum_{T \neq T_0 \wedge p \neq p_0} R_e^m(T, p) + \sum_T \sum_{p=1}^{|T|-m+1} f(e, T, p, m)} \\
&= \frac{2X}{mY + Z}
\end{aligned} \tag{14}$$

where

$$X = \sum_{\substack{T \neq T_0 \\ p \neq p_0}} R_e^m(T, p) \cdot f(e, T, p, m), \quad Y = \sum_{\substack{T \neq T_0 \\ p \neq p_0}} R_e^m(T, p), \quad Z = \sum_T \sum_{p=1}^{|T|-m+1} f(e, T, p, m)$$

Let $\widetilde{R_e^m}$ be a copy of R_e^m except that $R_e^m(T, p) = 1$; all other routing decisions remain the same. Then the F_1 score of the new segment router will be

$$\begin{aligned}
F_1(\widetilde{R_e^m}) &= \frac{2 \sum_T \sum_{p=1}^{|T|-m+1} \widetilde{R_e^m}(T, p) \cdot f(e, T, p, m)}{\sum_T \sum_{p=1}^{|T|-m+1} [m \cdot \widetilde{R_e^m}(T, p) + f(e, T, p, m)]} \\
&= \frac{2[X + f(e, T_0, p_0, m)]}{m(Y + 1) + Z} \\
&= \frac{(mY + Z) \cdot F_1(R_e^m) + 2f(e, T_0, p_0, m)}{m(Y + 1) + Z} \\
&= \frac{(mY + Z) \cdot F_1(R_e^m) + m \cdot [2f(e, T_0, p_0, m)/m]}{(mY + Z) + m}
\end{aligned} \tag{15}$$

which is a weighted mean of $F_1(R_e^m)$ and $2f(e, T_0, p_0, m)/m$ with weights $mY + Z$ and m . Note that $Z = \sum_T \sum_{p=1}^{|T|-m+1} f(e, T, p, m) \geq 0$, and $Z = 0$ if and only if $f(e, T, p, m) = 0$ for all T and p . If $Z = 0$, then e is inactive everywhere and $F_1(R_e^m) = 0$ for any R_e^m [‡], thus $\text{SRP}(e, m) = 0$

[‡]If $Y = R_e^m(T, p) = 0$ for all T and p , then $F_1(R_e^m)$ is undefined, which we do not concern.

and we can simply let $\alpha_e^m = 0$. Therefore, we assume that $Z > 0$, then both m and $mY + Z$ are positive. Hence, $F_1(\widehat{R_e^m}) \geq F_1(R_e^m)$ if and only if $2f(e, T_0, p_0, m)/m \geq F_1(R_e^m)$. Equality is achieved when and only when all equalities hold.

The above result indicates that, in order to increase $F_1(R_e^m)$, for any segment satisfying $R_e^m(T, p) = 0$ and $f(e, T, p, m) \geq (m/2) \cdot F_1(R_e^m)$, we should change the routing decision to $R_e^m(T, p) = 1$ [§], and for any segment satisfying $R_e^m(T, p) = 1$ and $f(e, T, p, m) < (m/2) \cdot F_1(R_e^m)$, we should change the routing decision to $R_e^m(T, p) = 0$. Under the case where the number of possible inputs is finite (which is the case for most LLMs due to their limited context windows), this will eventually result in a $\widehat{R_e^m}$ that activates and only activates all segments with $f(e, T, p, m) \geq (m/2) \cdot F_1(\widehat{R_e^m})$, whose F_1 cannot increase further. Such $\widehat{R_e^m}$ must be unique and maximizing $F_1(R_e^m)$: Otherwise, if there exists another $\widehat{R_e^m}'$ with $F_1(\widehat{R_e^m}') > F_1(\widehat{R_e^m})$, then the only segments where $\widehat{R_e^m}$ and $\widehat{R_e^m}'$ disagree are the ones satisfying $(m/2) \cdot F_1(\widehat{R_e^m}) \leq f(e, T, p, m) < (m/2) \cdot F_1(\widehat{R_e^m}')$, where $\widehat{R_e^m}(T, p) = 1$ and $\widehat{R_e^m}'(T, p) = 0$; however, changing $\widehat{R_e^m}$ on these segments to 0 should not increase $F_1(\widehat{R_e^m})$, thus $F_1(\widehat{R_e^m}') \leq F_1(\widehat{R_e^m})$, a contradiction. Therefore, we can let $\alpha_{e,m} = \lceil F_1(\widehat{R_e^m}) \rceil$, which yields Equation 4. $\square$

D Experiment setup details

D.1 Model architecture list

Table 4 lists the detailed architecture and configuration of all models where we conduct our experiments.

Table 4: Model architecture and configuration, sorted by model size. Experts: T: total; A: active; S: shared (not included in total).

Model	# Params (B)		# Layers	MoE Layer	# Experts		
	Total	Active			T	A	S
PowerMoE-3B (Shen et al., 2024c)	3.30	0.88	32	all	40	8	0
LLaMA-MoE-v1-3.5B (Zhu et al., 2024)	6.74	3.50	32	all	16	4	0
OLMoE-1B-7B-0125 (Muennighoff et al., 2025)	6.92	1.28	16	all	64	8	0
SwitchTransformers-Base-128 (Fedus et al., 2022)	7.42	0.22	24	every 2	128	1	0
LLaMA-MoE-v2-3.8B (Qu et al., 2024)	8.03	3.80	32	all	8	2	0
JetMoE-8B (Shen et al., 2024b)	8.52	2.33	24	all	8	2	0
OpenMoE-8B (Xue et al., 2024a)	11.86	3.80	24	every 6	32	2	1
MiniCPM-MoE-8x2B (Hu et al., 2024)	13.87	4.32	40	all	8	2	0
Qwen1.5-MoE-A2.7B (Qwen Team, 2024)	14.32	2.69	24	all	60	4	4
DeepSeek-V2-Lite (DeepSeek-AI et al., 2024a)	15.71	2.66	27	after 1st	64	6	2
DeepSeekMoE (Dai et al., 2024)	16.38	2.83	28	after 1st	64	6	2
XVERSE-MoE-A4.2B (XVERSE Technology Inc., 2024)	25.78	4.23	28	all	64	6	2
Qwen3-30B-A3B (Yang et al., 2025)	30.53	3.35	48	all	128	8	0
Yuan2.0-M32 (Wu et al., 2024)	39.94	3.70	24	all	32	2	0
Phi-3.5-MoE (Abdin et al., 2024)	41.87	6.64	32	all	16	2	0
GRIN-MoE (Liu et al., 2024b)	41.87	6.64	32	all	16	2	0
Mixtral-8x7B-v0.1 (Jiang et al., 2024)	46.70	12.88	32	all	8	2	0
Jamba-Mini-1.6 (Lenz et al., 2025)	51.57	12.11	32	every 2	16	2	0
NLLB-MoE-54B (NLLB Team et al., 2022)	54.50	3.75	48	every 4	128	2	0
Qwen2-57B-A14B (Yang et al., 2024a)	57.41	14.25	28	all	64	8	8

[§]When $f(e, T, p, m) = (m/2) \cdot F_1(R_e^m)$, changing $R_e^m(T, p)$ does not affect $F_1(R_e^m)$.

A few notes:

- SwitchTransformers-Base-128 and NLLB-MoE-54B are encoder-decoder models that use the T5 architecture. SwitchTransformers-Base-128 has 12 encoder layers and 12 decoder layers. NLLB-MoE-54B has 24 encoder layers and 24 decoder layers.
- JetMoE-8B employs mixture-of-attention(Shen et al., 2024a), which we keep intact in our experiments.
- GRIN-MoE shares the same architecture with Phi-3.5-MoE, but is trained using different methods.
- Jamba-Mini-1.6 employs a hybrid SSM-Transformer structure, yet the MoE part is identical to vanilla transformer-based MoE models.

D.2 Data processing and input generation

We first extract samples from RedPajama and downstream application datasets in plain text format. For RedPajama, this is already done. For LMArena, each of the original instances consists of two human-LLM conversations and a preference vote. We keep the instances where one of the conversations is preferred and concatenate all rounds from the preferred conversation (each round with its role and content) into one document. For OpenMath, OpenCode, and OpenScience, we simply concatenate the input and output of each instance.

After collecting samples from each RedPajama category and the downstream application dataset, we concatenate the samples within the domain, cutting them into input sequences of 512 tokens (the context window size of SwitchTransformers). We sample 2,048 input sequences for each domain, resulting in 22,528 input samples in total.

For SwitchTransformers, since the model is trained for masked language modeling, we randomly select 64 tokens from each input sequence, masking them in the original sequence as the encoder input and constructing the corresponding label sequence as the decoder input. For NLLB-MoE, as the model is trained for machine translation, we use the same sequence (with the English language token prepended) as both the encoder input and the decoder input. All other models do not need further data preprocessing, as they are decoder-only and trained for next token prediction.

E Additional results

E.1 Base vs. post-trained

We selected three models—LLaMA-MoE-v1, OLMoE, and JetMoE—that have both base and post-trained versions released, and calculated $\text{SRP}(E, m)$ and $\rho(E, m)$ of each version. Table 5 lists the results, from which we can see that the differences of both $\text{SRP}(E, m)$ and $\rho(E, m)$ between models before and after post-training are not significant enough to change the degree of local routing consistency, regardless of what type of post-training (SFT, DPO, etc.) is applied. Another related fact is that Phi-MoE-3.5 and GRIN-MoE, which share the same model architecture but are trained differently, have similar local routing consistency. Both indicate that the training method may be less important than the model architecture concerning local routing consistency.

Table 5: $\text{SRP}(E, m)$ between models before and after post-training.

Model	$m = 4$		$m = 16$		$m = 64$		$m = 256$	
	SRP	$\rho(E, m)$	SRP	$\rho(E, m)$	SRP	$\rho(E, m)$	SRP	$\rho(E, m)$
LLaMA-MoE-v1	55.78	1.03	45.29	2.39	41.61	2.92	40.62	3.52
+SFT	+0.01	-0.00	-0.01	-0.00	-0.01	+0.00	-0.00	-0.00
OLMoE	64.69	1.00	50.91	1.06	45.53	1.21	42.64	1.19
+SFT	+0.40	+0.00	+0.47	+0.01	+0.50	+0.02	+0.60	-0.02
+DPO	+0.37	+0.00	+0.43	+0.01	+0.47	+0.02	+0.57	-0.02
+Instruct	+0.45	+0.00	+0.56	+0.01	+0.62	+0.02	+0.74	-0.02
JetMoE	60.22	1.09	47.45	2.26	42.78	2.69	41.09	3.15
+SFT	-0.20	-0.00	-0.14	+0.01	-0.12	+0.02	-0.09	+0.03
+Chat	-0.20	-0.00	-0.15	+0.01	-0.13	+0.02	-0.10	+0.03

E.2 SRP and corpus perplexity

Figure 6 compares each model’s segment routing best performance with its mean log perplexity on S . Most models have corpus perplexity close to each other, and we do not find a significant relation between SRP and corpus perplexity.

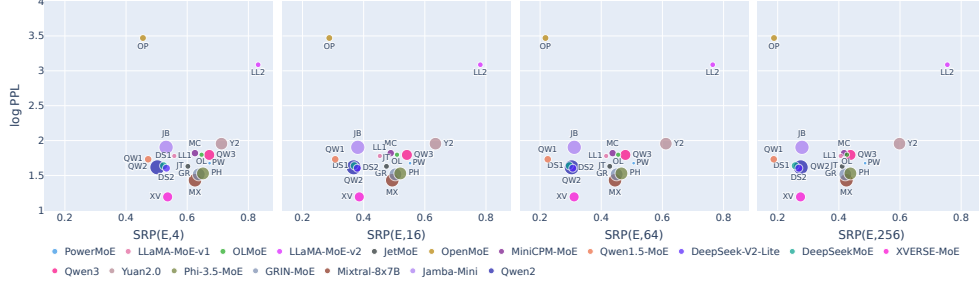


Figure 6: $SRP(E, m)$ of each model on S , compared with the mean log perplexity on S . Marker size represents model size.

E.3 SRP and model architecture

Table 6 compares each model’s segment routing best performance with several architecture parameters listed in Table 4 but not in Table 1. On these parameters, we do not observe significant patterns related to SRP.

Table 6: $SRP(E, 16)$ of each model on S , compared with architecture parameters listed in Table 4 but not in Table 1.

Model	SRP	# Params (B)		Active Param Ratio (%)	# Layers	# Experts	
		Total	Active			T	A
LLaMA-MoE-v2	78.16	8.03	3.80	47.36	32	8	2
Yuan2.0	63.48	39.94	3.70	9.27	24	32	2
PowerMoE	55.17	3.30	0.88	26.76	32	40	8
Qwen3	54.14	30.53	3.35	10.98	48	128	8
Phi-3.5-MoE	51.98	41.87	6.64	15.86	32	16	2
OLMoE	50.91	6.92	1.28	18.53	16	64	8
GRIN-MoE	50.39	41.87	6.64	15.86	32	16	2
Mixtral-8x7B	49.36	46.70	12.88	27.58	32	8	2
MiniCPM-MoE	48.85	13.87	4.32	31.13	40	8	2
JetMoE	47.45	8.52	2.33	27.36	24	8	2
LLaMA-MoE-v1	45.29	6.74	3.50	51.85	32	16	4
XVERSE-MoE	38.58	25.78	4.23	16.39	28	64	6
Jamba-Mini	38.08	51.57	12.11	23.48	32	16	2
DeepSeek-V2-Lite	37.92	15.71	2.66	16.94	27	64	6
DeepSeekMoE	36.94	16.38	2.83	17.27	28	64	6
Qwen2	36.74	57.41	14.25	24.82	28	64	8
NLLB-MoE (en)	25.24	27.25	1.88	6.88	24	128	2
(de)	31.35	27.25	1.88	6.88	24	128	2
Qwen1.5-MoE	30.71	14.32	2.69	18.78	24	60	4
OpenMoE	28.77	11.86	3.80	32.07	24	32	2
SwitchTF (en)	19.33	3.71	0.11	3.02	12	128	1
(de)	19.27	3.71	0.11	3.02	12	128	1

E.4 SRP per segment position

To determine whether the segment position p can affect the segment routing best performance, we calculate $SRP(E, m)$ on each segment position by summarizing statistics of all segments that share

the same position. Figure 7 illustrates this position-wise $\text{SRP}(E, m)$ at each possible segment position. Most models have nearly constant $\text{SRP}(E, m)$ at every position except $p = 0$, where many models activate specialized experts to handle the beginning of the input sequence. This stability of local routing consistency across input positions allows us to use segments from all positions to calculate $\text{SRP}(E, m)$, and apply conclusions based on $\text{SRP}(E, m)$ to any segment of the input (except the very first one).

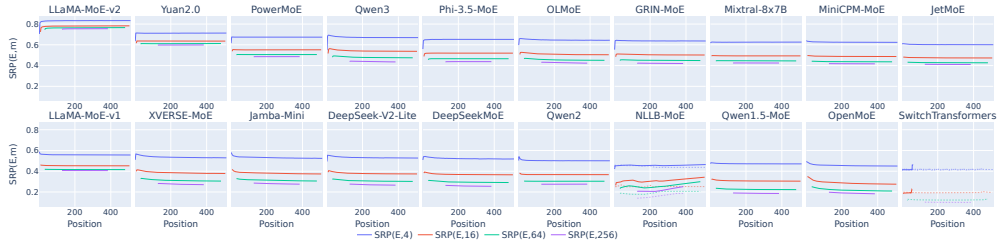


Figure 7: Position-wise SRP of each model on S . For encoder-decoder models, dotted lines show the encoder $\text{SRP}(E, m)$ and solid lines show the decoder ones.

E.5 SRP across domains

To verify whether local routing consistency is transitive across different domains, we calculate the correlation of expert segment routing best performance between pair-wise domains and demonstrate it in Figure 8. We also compute the correlation of expert activation frequency between pair-wise domains, results illustrated in Figure 9. By comparing corresponding heatmaps, we can see that local routing consistency is nearly always positively correlated, even between distant domains on which the experts’ activation frequencies are negatively correlated. This means that local routing consistency is transitive; domain-specialized experts with high local routing consistency in one domain tend to exhibit it in any other domain. We also found that some models (e.g., LLaMA-MoE-v2 and Qwen2) do not show a significant difference between domains, which is aligned with the results in Section 4.2.

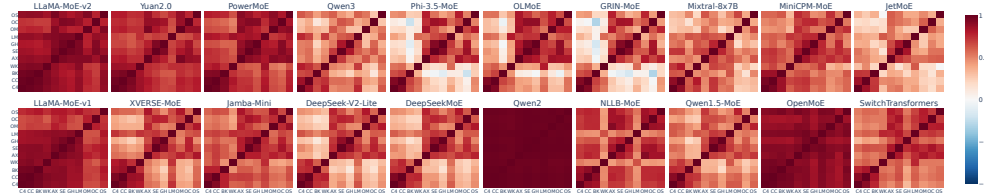


Figure 8: Correlation between domain-wise expert SRP of each model. C4: C4; CC: CommonCrawl; BK: Books; WK: Wikipedia; AX: ArXiv; SE: StackExchange; GH: GitHub; LM: LMArena; OM: OpenMath; OC: OpenCode; OS: OpenScience.

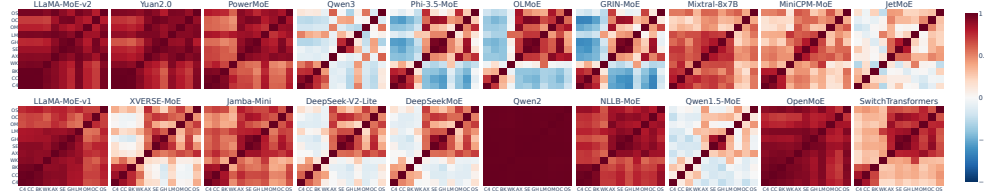


Figure 9: Correlation between the domain-wise expert activation frequency of each model. C4: C4; CC: CommonCrawl; BK: Books; WK: Wikipedia; AX: ArXiv; SE: StackExchange; GH: GitHub; LM: LMArena; OM: OpenMath; OC: OpenCode; OS: OpenScience.

E.6 SRP vs. SCH

To clarify the relation between $\text{SRP}(E, m)$ and $\text{SCH}(E, m, \rho)$, Table 7 lists the correlation between them across all models. $\text{SRP}(E, m)$ and $\text{SCH}(E, m, \rho)$ are always highly positively correlated regardless of the values of m and ρ . This ensures that SCH shares the same property of SRP under reasonable segment length and cache size. Furthermore, when ρ is around 1.5, the two metrics are most closely related, nearly perfectly linear, aligned with Figure 2 where most models have $\rho(E, m)$ between 1–3 when $m \geq 16$, as well as our previous claim that $\rho = 2$ balances cache effectiveness and efficiency.

Table 7: Correlation between $\text{SRP}(E, m)$ and $\text{SCH}(E, m, \rho)$ across all models. Bold font indicates the highest correlation across ρ for each m .

m	$\rho = 0.5$	$\rho = 1.0$	$\rho = 1.5$	$\rho = 2.0$	$\rho = 2.5$	$\rho = 3.0$
4	96.58	97.98	98.21	96.04	89.07	75.05
16	97.87	99.52	99.76	98.49	95.70	92.33
64	96.75	99.01	99.78	99.02	96.81	93.60
256	95.41	98.32	99.58	99.20	97.11	93.70

E.7 Statistical significance

Due to the very high correlation between SRP and SCH, we choose SCH to represent local routing consistency, and report the 95% confidence intervals when $\rho = 2$ in Table 8. The confidence intervals are obtained by bootstrapping 1,000 times with samples from S .

Table 8: 95% confidence interval of $\text{SCH}(E, m, 2)$ of each model. The decoder of SwitchTransformer does not have valid data when $m = 256$.

Model	$m = 4$	$m = 16$	$m = 64$	$m = 256$
LLaMA-MoE-v2	(99.93, 99.94)	(98.51, 98.56)	(97.36, 97.45)	(96.69, 96.80)
Yuan2.0	(94.99, 95.10)	(82.39, 82.57)	(78.44, 78.64)	(76.57, 76.81)
PowerMoE	(92.85, 92.95)	(80.13, 80.30)	(74.53, 74.75)	(71.92, 72.18)
Qwen3	(91.58, 91.74)	(75.48, 75.75)	(67.03, 67.40)	(61.62, 62.09)
Phi-3.5-MoE	(89.48, 89.70)	(74.25, 74.62)	(67.10, 67.57)	(63.26, 63.84)
OLMoE	(89.23, 89.44)	(72.69, 73.04)	(65.09, 65.52)	(61.04, 61.54)
GRIN-MoE	(88.18, 88.40)	(72.47, 72.83)	(65.07, 65.54)	(61.07, 61.63)
Mixtral-8x7B	(88.20, 88.28)	(73.89, 74.00)	(66.25, 66.41)	(62.20, 62.40)
MiniCPM-MoE	(88.11, 88.19)	(73.09, 73.19)	(64.95, 65.07)	(60.73, 60.87)
JetMoE	(85.64, 85.72)	(70.79, 70.88)	(63.04, 63.16)	(59.05, 59.20)
LLaMA-MoE-v1	(80.80, 80.89)	(66.96, 67.04)	(60.25, 60.35)	(57.07, 57.18)
XVERSE-MoE	(78.17, 78.31)	(57.34, 57.54)	(46.39, 46.61)	(41.00, 41.25)
Jamba-Mini-1.6	(77.82, 77.95)	(56.87, 57.05)	(46.43, 46.68)	(41.10, 41.41)
DeepSeek-V2-Lite	(77.86, 78.01)	(56.62, 56.82)	(45.76, 46.00)	(40.15, 40.43)
DeepSeekMoE	(76.99, 77.14)	(55.21, 55.42)	(44.07, 44.32)	(38.37, 38.65)
Qwen2	(75.18, 75.26)	(54.86, 54.92)	(45.26, 45.32)	(40.96, 41.03)
NLLB-MoE (en)	(62.31, 62.44)	(37.60, 37.84)	(28.56, 28.86)	(24.06, 24.37)
(de)	(65.38, 65.59)	(44.93, 45.24)	(37.69, 38.03)	(31.76, 32.12)
Qwen1.5-MoE	(69.07, 69.20)	(45.31, 45.48)	(33.37, 33.58)	(27.49, 27.72)
OpenMoE	(65.03, 65.24)	(42.07, 42.39)	(32.22, 32.60)	(27.65, 28.09)
SwitchTF (en)	(54.81, 54.95)	(26.80, 27.04)	(17.44, 17.72)	(13.66, 13.97)
(de)	(55.39, 55.56)	(26.76, 27.06)	(16.64, 16.94)	

E.8 Layer level results

Figure 10 illustrates each model’s layer-wise SRP. Most models have peak SRPs among middle layers, while some (e.g., Yuan2.0 and MiniCPM) have another peak at the last layer. We conjecture that middle layers are less tied to input/output tokens and thus more sensitive to the general topic, and

the final layers process highly abstract information that is also more related to the overall topic. Both encourage routers to select similar experts within a local segment that share the same topic across tokens. PowerMoE and Qwen2 have another peak on layer 2 due to expert imbalance. Appendix E.9 gives a clear view on this.

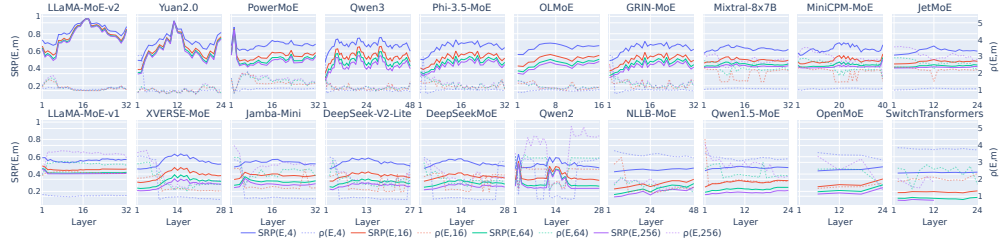


Figure 10: Layer-wise SRP on S of each model. Solid lines show $\text{SRP}(E, m)$ while dotted lines show corresponding $\rho(E, m)$.

We also calculated layer-wise SCH, results demonstrated in Figure 11. The patterns are the same as SRP, indicating a high correlation between the two metrics.

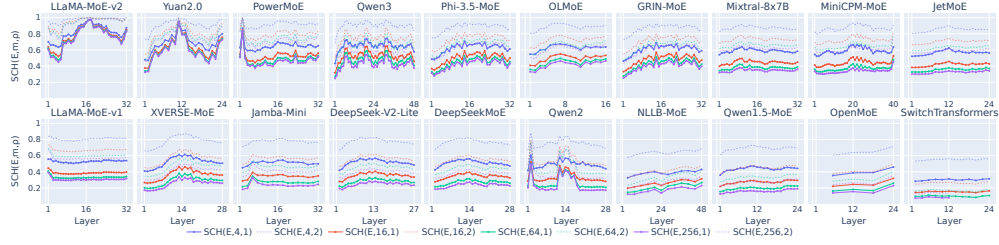


Figure 11: Layer-wise SCH on S of each model. Solid lines show $\text{SCH}(E, m, 1)$ while dotted lines show corresponding $\text{SCH}(E, m, 2)$.

E.9 Expert level results

We demonstrate expert-wise segment routing best performance against activation frequency in Figure 12. LLaMA-MoE-v2, Yuan2.0, and PowerMoE have experts with very high activation frequency. These experts naturally have very high local routing consistency and contribute to these models' high model-level local routing consistency. The imbalanced experts of PowerMoE mainly belong to layer 2, which also explains the observation in Section E.8.

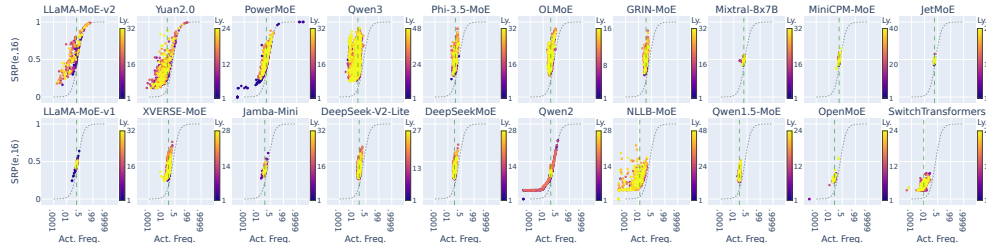


Figure 12: Per-expert activate frequency vs. SRP. The x-axis is stretched to show experts with very low or high activation frequency. Gray dashed lines indicate the theoretical lower bound of SRP at different activation frequencies. Green dashed lines show the expected activation frequency of experts from each model.

Furthermore, Figures 13, 14, 15 and 16 compares SRP with domain and vocabulary specializations. The plots are aligned with the conclusion of Section 4.2 that when the model exhibits domain

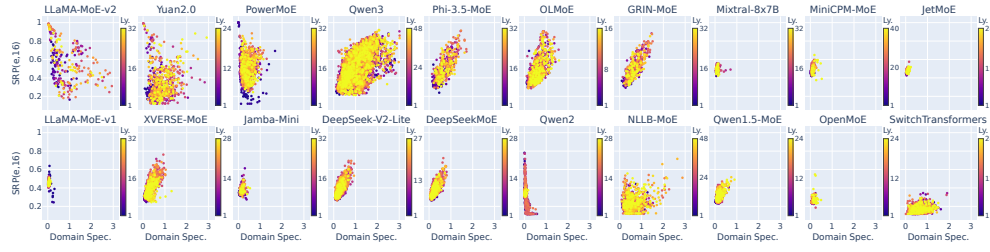


Figure 13: Per-expert domain specialization vs. SRP.

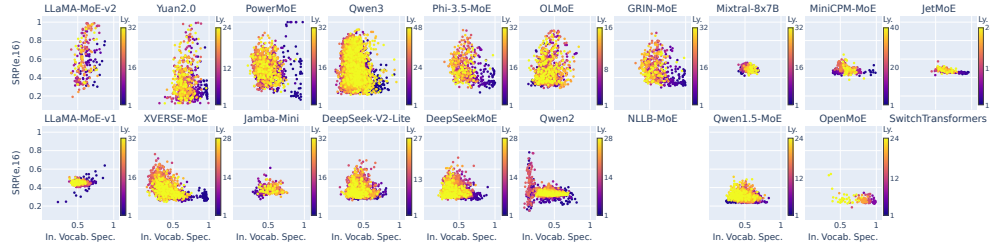


Figure 14: Per-expert input vocabulary specialization vs. SRP. Encoder-decoder models are not involved due to different input formats from other decoder-only models.

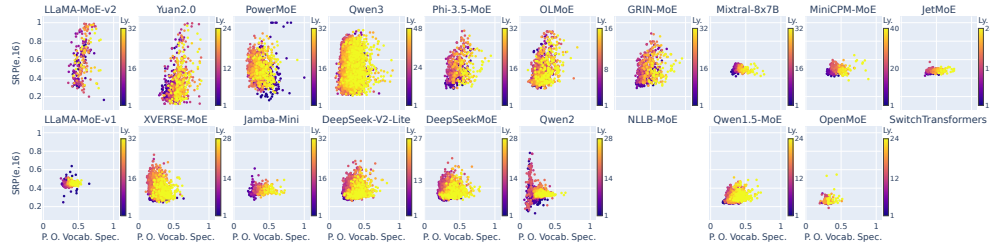


Figure 15: Per-expert predicted output vocabulary specialization vs. SRP. Encoder-decoder models are not involved due to different input formats from other decoder-only models.

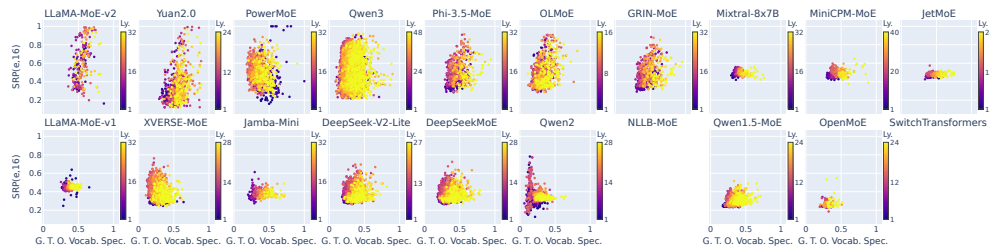


Figure 16: Per-expert ground-truth output vocabulary specialization vs. SRP. Encoder-decoder models are not involved due to different input formats from other decoder-only models.

specialization, domain-specialized experts contribute more to overall local routing consistency than vocabulary-specialized experts.